

Agentic Research Assistant

Project Specification Document · AMD AI DevMaster Hackathon — Track 2: Agentic AI

Submitted by: Yash Vardhan

GitHub: github.com/Yashhh0602/agentic-research-amd-scaffold

1. Application Scenarios

This project is a local knowledge assistant that answers technical research questions by combining retrieval-augmented generation (RAG) over a curated knowledge base with tool use (web search, sandboxed code execution). The demonstrated scenario is developer support: answering questions about AMD ROCm/HIP documentation, e.g. explaining API differences between CUDA and HIP, and guiding kernel porting. The same architecture generalizes to any local-knowledge-base assistant use case: enterprise documentation Q&A, internal engineering copilots, or research-paper assistants, since the retrieval and orchestration layers are domain-agnostic.

The core design question this project explores is not just “can an agent answer a question” but “how should multiple agents with different jobs share a single GPU efficiently.” A naive pipeline that runs every agent strictly one after another leaves the GPU idle while non-GPU steps (like a database query) run, and wastes wall-clock time when two agents have no actual dependency on each other. This project treats agent scheduling itself as an optimization target, not just single-model inference speed.

2. Agent Architecture

Four agents, each with a distinct job and a distinct dependency on the others:

Agent	Job	Depends on	Model
Retriever	Semantic search over a pgvector knowledge base	Query only	None (DB-only)
Reasoning	Decomposes the query into sub-tasks, plans tool use	Query only	Small (3B)
Executor	Calls tools: web search (Tavily), sandboxed code execution	Reasoning's plan	None / small
Synthesizer	Combines everything into a final, cited answer	Everything	Large (7B, AWQ)

Dependency graph and execution modes:

Retriever and Reasoning both depend only on the original query, not on each other's output, so they can run concurrently. Executor requires Reasoning's plan before it can start. Synthesizer requires the outputs of both Retriever and Executor before it can write anything. This is a real constraint derived from data dependencies in the orchestrator code, not an arbitrary design choice.

```
SEQUENTIAL (naive):
Retriever -> Reasoning -> Executor -> Synthesizer
(each agent idles while the previous one runs)

CONCURRENT (optimized):
Retriever ---+
Reasoning ---+
Executor ---+
Synthesizer ---+
```

```
    +--> Executor -> Synthesizer
Reasoning ---+
(Retriever + Reasoning run in parallel; downstream dependencies respected)
```

Both execution modes are implemented on the same orchestrator code path and exposed as a live toggle in the UI (not a config flag read once at startup), and both stream agent events over a WebSocket to a live 4-lane trace panel in the frontend, so the same code path drives the demo visualization and the benchmark measurements.

3. Introduction to Core Capabilities

Retrieval-augmented generation

PostgreSQL + pgvector cosine similarity search over a curated AMD ROCm/HIP documentation knowledge base: 7 source pages (HIP porting guide, HIP programming model, and memory management docs — coherence control, host/device/unified/virtual memory), ingested into 259 chunks. Chosen specifically because the hero demo query splits into two genuinely independent sub-questions, good material for demonstrating real concurrent agent execution rather than an artificially split single question.

Tool use

Executor agent calls Tavily for live web search when the knowledge base doesn't cover a query, and can run sandboxed Python code execution (subprocess-isolated, resource-limited via rlimit on Linux, stripped environment, hard wall-clock timeout) for computational sub-tasks.

Mixed model-size routing

Reasoning (routing/planning) runs on a small 3B model (Qwen2.5-3B-Instruct). Synthesizer, which produces the final answer the user reads, runs on a separate, larger 7B model (Qwen2.5-7B-Instruct-AWQ) served as its own independent vLLM process on the same GPU, with its own memory allocation. This is a real architectural split, not a config label: an earlier version routed both sizes to the same 3B model, which caused the Synthesizer to introduce ungrounded facts not present in retrieved context in roughly half of repeated test runs on an identical query, despite strict system prompting and low temperature. Retrieval itself was independently verified correct via a standalone check before concluding the issue was a model capability ceiling; switching Synthesizer to the 7B model fixed it, producing consistently clean, fully-cited answers across repeated runs.

Live multi-agent trace panel

The Next.js frontend streams agent events (started / tool_call / tool_result / finished) over a WebSocket, rendering four live lanes so a viewer can watch, in real time, which agents are running, and whether they're running sequentially or concurrently.

Dependency-aware orchestration

The orchestrator (backend/app/core/orchestrator.py) implements both execution modes on one shared code path using asyncio, with a real producer/consumer event queue for the concurrent branch so that Retriever and Reasoning's events interleave live rather than being buffered and released in batch.

4. Model Introduction & Local Deployment Plan

Two models are deployed as two independent vLLM processes on the AMD Radeon Cloud pod (ROCm 7.2.1, vLLM 0.16.0), each with its own GPU memory allocation so they run simultaneously without contention:

- **Qwen2.5-3B-Instruct** — served on port 8000, --gpu-memory-utilization 0.25. Used by Retriever (embeddings, via local Ollama for CPU-cheap embedding calls) and Reasoning (query decomposition / planning).
- **Qwen2.5-7B-Instruct-AWQ** — served on port 8001, --gpu-memory-utilization 0.5. Used exclusively by Synthesizer for final answer generation.

The LLM backend is swappable via a single environment variable (LLM_BACKEND): o l l a m a for local CPU-only development and logic testing (no GPU required, used during early iteration before spending pod GPU-hours), and v l l m for real GPU-backed inference against the AMD pod, reached over an SSH tunnel from the local development machine. Embeddings always run locally via Ollama regardless of backend, since they are cheap and CPU-friendly, and there is no reason to route them through the pod.

5. Optimization for Inference Speed on AMD Radeon GPU

Two independent benchmarks were run against the live vLLM endpoint on the AMD ROCm pod, measuring two different kinds of concurrency:

Benchmark	Baseline	Optimized	Speedup
Agent-level (single query, sequential vs. concurrent orchestration)	21.401s	18.363s	1.17x
Throughput (5 simultaneous users, one-at-a-time vs. concurrent)	125.59s	30.82s	4.08x

The agent-level result is intentionally modest: only two of four agents (Retriever, Reasoning) have no dependency on each other, so there is a hard, by-design ceiling on how much orchestration-level concurrency can win — this is not an implementation gap. The throughput result is the stronger number and reflects vLLM's continuous batching doing real work on ROCm: it does not mean any single user's query got faster under load (GPU compute is shared across simultaneous requests), it means the system serves more total requests per minute, which is the metric that matters for real multi-user deployment. Both numbers were measured with scripts/benchmark.py and scripts/throughput_benchmark.py directly against the pod, logged as timestamped JSON in proofs/ with rocm-smi output where available, so results are reproducible rather than hand-typed.